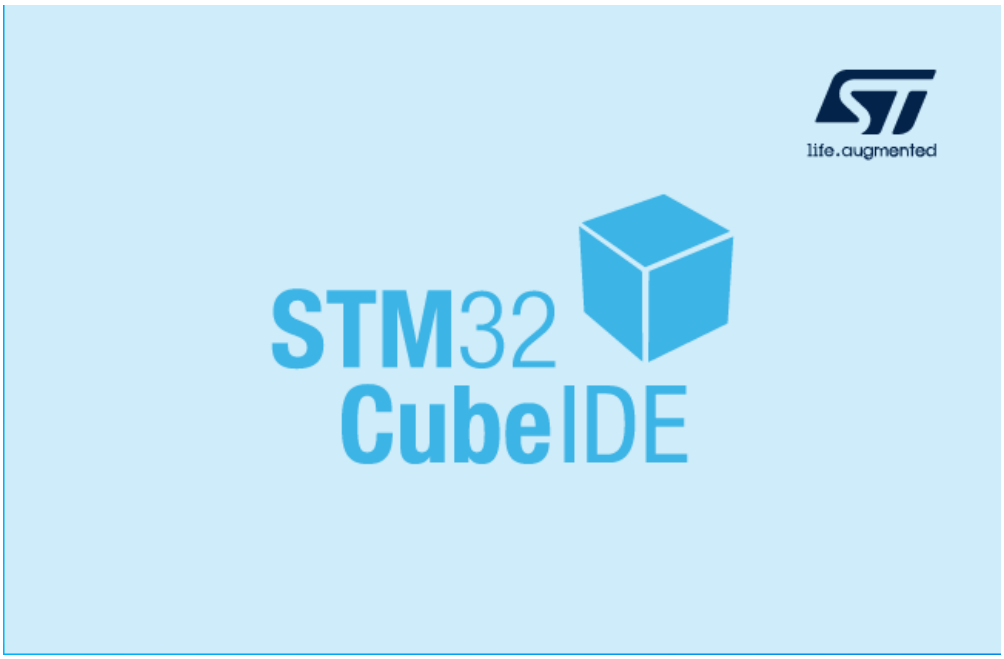
Лабораторная работа №1.
Ознакомление со средой программирования STM32CubeIDE

Цель работы: познакомиться с основами работы со средой программирования микроконтроллеров STM32 STM32CubeIDE и режимом отладки.

Общие сведения:

STM32CubeIDE — это усовершенствованная платформа разработки C/C++ с функциями настройки периферийных устройств, генерации кода, компиляции кода и отладки для микроконтроллеров и микропроцессоров STM32. Она основана на платформе Eclipse®/CDT™ и наборе инструментов GCC для разработки и GDB для отладки. Она позволяет интегрировать сотни существующих плагинов, дополняющих возможности Eclipse® IDE.



STM32CubeIDE объединяет функции настройки STM32 и создания проектов из STM32CubeMX, предлагая универсальный инструмент и экономя время на установку и разработку. После выбора пустого MCU или MPU STM32 или предварительно сконфигурированного микроконтроллера или микропроцессора из выбора платы или выбора примера создается проект и генерируется код инициализации. В любой момент разработки пользователь может вернуться к инициализации и настройке периферийных устройств или промежуточного программного обеспечения и повторно сгенерировать код инициализации, не влияя на пользовательский код.

STM32CubeIDE включает в себя анализаторы сборки и стека, которые предоставляют пользователю полезную информацию о состоянии проекта и требованиях к памяти.

STM32CubeIDE также включает стандартные и расширенные функции отладки, включая просмотр регистров ядра ЦП, памяти и регистров периферийных устройств, а также просмотр переменных в реальном времени, интерфейс Serial Wire Viewer или анализатор неисправностей.

Загрузка STM32CubeIDE

Перед тем, как начать работать с микроконтроллерами STM32, необходимо загрузить и установить среду программирования STM32CubeIDE

Данный программный комплекс можно загрузить на сайте: https://www.st.com/en/development -tools/ stm32cubeide.html

Перейдя в раздел Get Software, можно будет увидеть несколько вариантов дистрибутива STM32CubeIDE для различных операционных систем, в том числе и для Windows.

Загрузку среды можно осуществить, нажав кнопку Get Latest (при желании, в разделе Select version можно загрузить версию помладше на одну-две версии).

Внимание: для загрузки среды STM32CubeIDE необходимо зарегистрировать аккаунт на сайте ST.com

Get Software

	Part Number	General Description	Latest version	Download	All versions
+	STM32CubeIDE-DEB	STM32CubeIDE Debian Linux Installer	1.17.0	Get latest	Select version
+	STM32CubeIDE-Lnx	STM32CubeIDE Generic Linux Installer	1.17.0	Get latest	Select version
+	STM32CubeIDE-Mac	STM32CubeIDE macOS Installer	1.17.0	Get latest	Select version
+	STM32CubeIDE-RPM	STM32CubeIDE RPM Linux Installer	1.17.0	Get latest	Select version
+	STM32CubeIDE-Win	STM32CubeIDE Windows Installer	1.17.0	Get latest	Select version

После загрузки архива с установщиком, необходимо извлечь дистрибутив в любое место, затем запустить его установку.

Следуя инструкциям, установить STM32CubeIDE.

После успешной установки, можно запустить среду через соотвествующий ярлык на рабочем столе.

Задание:

1. Установить среду программирования STM32CubeIDE.
2. Создать проект STM32 с наименованием “nucleo_f103rb”.

3. Запустить режим отладки основной программы (main.c) в созданном проекте. Убедиться, что программа останавливается на строчке “while (1) {“
4. Добавить в раздел private user code следующие функции:

```
int add(int a, int b)
{
      return a + b;
}

int sub(int a, int b)
{
      return a - b;
}

int mul(int a, int b)
{
      return a * b;
}

int div(int a, int b)
{
      return a / b;
}
```

5. Добавить следующий код в раздел private typedef:

```
typedef enum {
      ADD,
      SUB,
      MUL,
      DIV
} MATH_OP;
```

6. Добавить следующий код в int main:

до цикла while (1) добавить

```
MATH_OP math_op= ADD;
int result = 0;
int isExit = 0;
```

в цикл while (1) добавить:

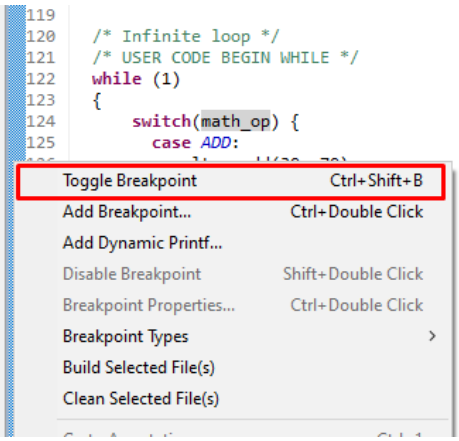
```
switch(math_op) {
```

```
        case ADD:
        result = add(30, 70);
        math_op= SUB;
        break;
        case SUB:
        result = sub(4000, 260);
        math_op= MUL;
        break;
        case MUL:
        result = mul(81, 9);
        math_op= DIV;
        break;
        case DIV:
        result = div(10000, 25);
        isExit = 1;
        break;
    }

    if (isExit) {
        break;
    }
```

7. Запустить добавленный код в режиме отладки.
8. Зафиксировать значения result на каждом вызове функции:
   - add;
   - sub;
   - mul;
   - div.

Примечание: чтобы отлаживаться в коде программы, необходимо ставить точки останова. Для этого, нажмите правой кнопкой мыши в синем столбце рядом с нужной строкой кода, а затем выберите опцию Toggle breakpoint для установки/удаления точки останова:



9. Зафиксировать, что программа выходит из главного цикла после вызова всех функций, на условии if (isExit) {break;}

   Важное примечание: добавленный код из шагов 4-6, при последующей генерации проекта **должен сохраниться в файле main.c!**

10. Представить результаты всех зафиксированных действий и значений при выполнении функций на шагах 3-9 в отчёте.